

# CPSC 375

## Introduction to Data Science and Big Data Analytics

Dr. Kanika Sood  
kasood@fullerton.edu

**Week 02 L02**

# Topics

- Why Python for data science?
- Basic Python libraries, operations and data structures

| Week   | Material                                                          |
|--------|-------------------------------------------------------------------|
| Week 1 | What is data science? Introduction to Python                      |
| Week 2 | Visualization                                                     |
| Week 3 | Exploratory data analysis                                         |
| Week 4 | Data wrangling                                                    |
| Week 5 | Tidy data                                                         |
| Week 6 | Linear modeling                                                   |
| Week 7 | Nearest neighbors classification<br>Distance measures             |
| Week 8 | K-means clustering<br>Evaluation metrics<br><b>Midterm Review</b> |
| Week 9 | Evaluation metrics (contd.)                                       |
|        | <b>Midterm exam</b>                                               |

# Statistical software

- SAS
- SPSS
- Matlab
- S-Plus
- R
- ...

# History of Python

- Object-oriented scripting language
  - extensible with C or C++
  - developed at Dutch National Research Institute
  - **“There should be one– and preferably only one– obvious way to do it.”** – Guido van Rossum, creator of Python
- Python in Data Science:
  - scientific computing
  - in 2006, first release of NumPy library
  - huge ecosystem: pandas, scikit-learn, statsmodels, ...



# Python: Main features

- General purpose programming language  
Object-oriented
- NumPy library implements vector-based operations
  - Similar to MATLAB
- Packaging: easily add software libraries from different sources
- And features of typical programming languages

# Python and statistics

- Many packages for statistics and data analysis
  - linear models and generalized linear models
  - nonlinear regression models
  - time series analysis
  - classical parametric and nonparametric tests
  - clustering
  - smoothing
- And newer statistical methods
  - many data scientists provide their methods as Python packages

# Python and Presentations

- Many functions for creating various types of data presentations
- Streamlit, Dash, Shiny
  - Python packages to build interactive web apps for data science
  - Host standalone apps on a webpage
  - Packages for general-purpose web development (Django, Flask, FastAPI)
- Jupyter Notebooks
  - Authoring framework for data science
  - A single Jupyter notebook file to both
    - save and execute code
    - generate reports
      - HTML
      - PDF
      - ...

# Strengths

- Free and Open Source
- Strong user community
- Maintained by experts
- Highly extensible
- Continuous improvement
- **Core language is easy to learn**
- Available on many platforms



# Weaknesses

- Slow for large datasets
- All data has to fit in memory (RAM)
  - Data limited to RAM size (e.g., 8GB)
  - Big data platforms also support Python
- Steep learning curve for libraries
  - NumPy, SciPy, Pandas, scikit-learn, statsmodels, matplotlib, plotly, letsplot, Tensorflow, PyTorch, ...
- Data type conversion between libraries
  - Python lists, NumPy arrays, Pandas DataFrames, ...

# Installation – Local Machine

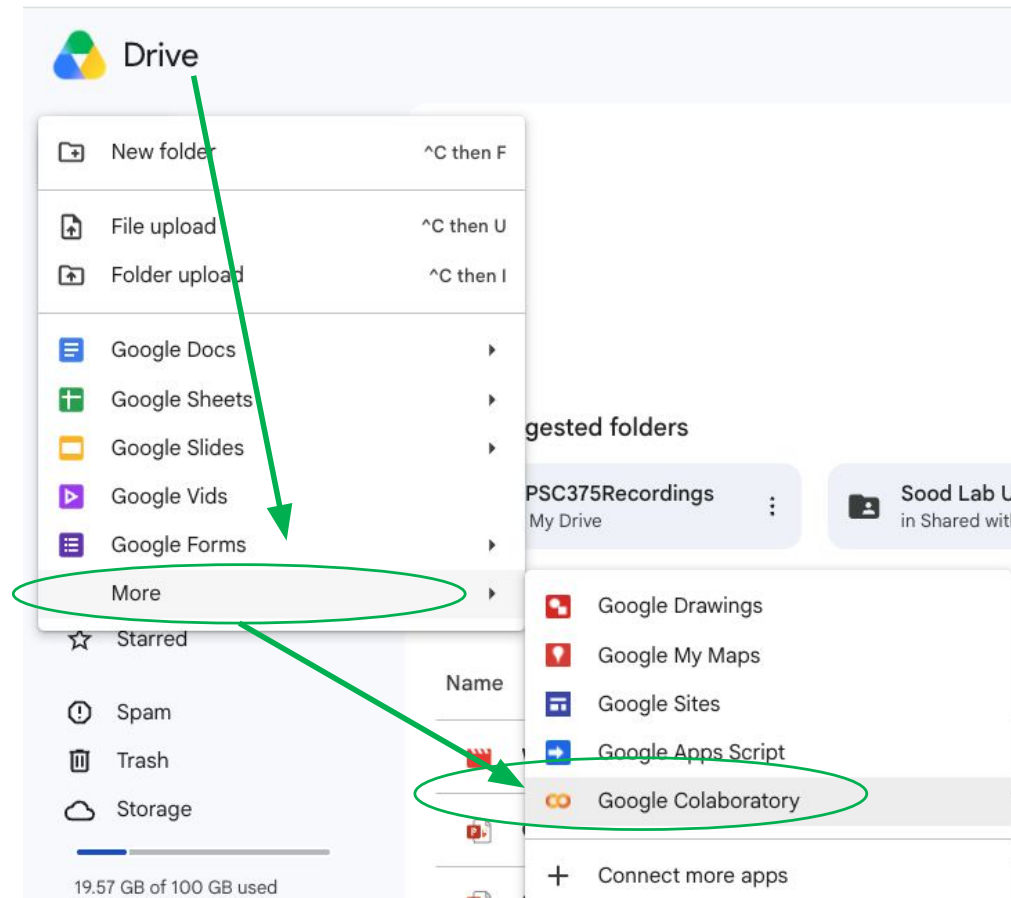
- Three steps
  1. Install the [Anaconda Distribution](#)
    - Interpreter, libraries, package manager
    - Binaries for Windows/Linux/MacOS
  2. Install [Visual Studio Code](#)
  3. Configure Visual Studio Code [Extensions](#)
    - See *Python for Data Science*, [Prerequisites](#) for details.



# Installation – Cloud



- Required for classwork
- Two steps
  1. Log into [Google Drive](#)
    - for classwork, use your @csu.fullerton.edu account
    - You may need to [Switch between multiple Google accounts](#)
  2. Choose *New* → *More* → *Google Colaboratory*



# Installation – Cloud



- Required for

- Two steps

## 1. Log into

- for cla

@csu.

accou

- You m

[Switch](#)

[multip](#)

[accou](#)

## 2. Choose M

→ Goog

Colabora

The screenshot displays the Google Workspace Marketplace interface. On the left, a sidebar shows the Google Drive interface with options like 'New folder', 'File upload', 'Folder upload', 'New project', and various Google apps (Docs, Sheets, Slides, Vids, Forms). The main area shows search results for 'Colaboratory'. The top bar includes the Google Workspace Marketplace logo and a search bar with 'Colaboratory' entered. Below the search bar, there are filters for 'All filters (1)', 'Works with Drive', and 'Price'. The search results section is titled 'Search results for Colaboratory' and includes a disclaimer: 'Google doesn't verify reviews or ratings. [Learn more about reviews and results](#)'. Three results are shown: 'Colaboratory' by the Colaboratory team, 'Translate Video' by Video Translator - Trans..., and 'Document translator' by Document Translator - ... Each result includes a star rating, download count, and an 'Install' button. Below these, there are more results: 'Chat with Any PDF', 'AI Video Editor', and 'Table OCR'.

# Getting help

Details about a specific module whose name you know (high-level overview, list of functions):

```
>>> import scipy.stats
>>> help(scipy.stats)
```

Details about a specific command whose name you know (input arguments, options, algorithm, results):

```
>>> help('scipy.stats.ttest_1samp')
Help on function ttest_1samp in scipy.stats:

scipy.stats.ttest_1samp = ttest_1samp(a, popmean, axis=0, nan_policy='propagate',
alternative='two-sided', *, keepdims=False)
    Calculate the T-test for the mean of ONE group of scores.
...
```

From a code cell in a Jupyter notebook:

```
import scipy.stats
?scipy.stats
```

# Arithmetic operations

- + (addition)
- - (subtraction)
- \* (multiplication)
- / (division)
- // (integer division)
- \*\* (exponentiation)

# Basic functions

- `abs(x)`

`from math import...`

- `log(x)`
- `log2(x)`
- `log10(x)`
- `exp(x)`
- `sqrt(x)`

# Assignment

```
□ a = 20 # assign 20 to object "a"  
□ a # print the value of object  
20
```



# Data types

- int
  - 124
- float
  - 3.141
- str
  - "strings" or 'strings'
- bool
  - True, False

# Data Structures in Python

|               | 1-dim                      | 2-dim                         |
|---------------|----------------------------|-------------------------------|
| All same type | <b>ndarray<br/>(NumPy)</b> | <b>matrix<br/>(NumPy)</b>     |
| Mixed types   | <b>list</b>                | <b>DataFrame<br/>(Pandas)</b> |

# Vectors

- **ndarray**: an ordered collection of data of the same type

```
import numpy as np
```

```
a = np.array([10, 20, 30])
```

- Subscripts in square brackets
  - `a[0]` is the first element in vector `a`

# Vectorized functions

- Most NumPy functions take an “array\_like” object as input
- Answer can be a vector or numeric
  - `np.sum(np.array([1, 2, 3]))`
- Most operators are vectorized too
  - `a*2`
  - `array([2, 4, 6])`
  - `np.array([True, False, True]) & np.array([True, True, False])`
  - `array([ True, False, False])`

# Sequences and Slices

- `np.arange(1, 4)`
  - `array([1, 2, 3])`
- `np.arange(1,4,2) == np.arange(1,4,step=2)`
- `np.linspace(1,20,num=8)`
- Can be used for numeric indexing
  - `a[np.arange(2,6)]` is elements at index 2-5 in vector `a`
  - “Slice” syntax `a[2:6]` as shortcut
- **Caution:**
  - Arrays begin at index 0
  - Last element is *stop* - 1

# Opening classwork



- <https://github.com/soodk6/cpsc375-classwork>
  - python.ipynb – Part 1

## Links on Canvas

- From GitHub
  - View notebook
  - Click *Open in Colab*
- From Google Colaboratory
  - Open notebook → GitHub
  - User *soodk6*, repository *soodk6/cpsc375-classwork*
  - View notebook



Image Courtesy: Gemini

# Submitting classwork



- Check that you are logged in with your `csu.fullerton.edu` email address
  - Click your profile photo or email address at the top right corner
    - Check that it shows your `@csu.fullerton.edu` email address at the top
  - If not, choose that account or click *Add Account*
- Click *Copy to Drive or File* → *Save a Copy in Drive*
- Rename the notebook with your groupmates' lastnames
  - E.g.: Doe-Chester-Hoffman.ipynb
  - Write the names and role of group members
  - Roles: **manager**, **recorder+presenter**, **timekeeper**
  - Add your answers
- Click *File* → *Move*
  - Change folder to *Shared with Me* → CPSC375StudentFolder-Term → today's date
- Will not be graded may be used for class participation points
  - Can easily share with rest of class

# Classwork - Suggested Workflow

- Multiple users can open the same notebook, but content sync is not as seamless as Google Docs
  - you may receive a notification that the notebook has changed, but you will need to reload the page.
- Suggestion for classwork:
  - *Recorder* opens notebook from GitHub and saves answers to Google Drive
  - Other group members open notebook from GitHub but do not save to Google Drive.
    - Experiment with a temporary notebook
    - Colab will say “Cannot save changes” until you make a copy.



# Lists and Dictionaries

- **list**: an **ordered** collection of data of **arbitrary** types.

```
□ course = ["CPSC", 375, False]
```

```
>>> course[0]  
'CPSC'
```

- **dict**: an **ordered** collection of key-value pairs.

```
□ course = dict(dept="CPSC", number=375, required=False)
```

or

```
□ course = {'dept':"CPSC", 'number':375, 'required':False}
```

```
□ course['dept']  
'CPSC'
```

```
□ course["number"]  
375
```

- Typically
  - list elements are accessed by their index (an integer)
  - dict elements by their key (usually a character string)

# Matrices and Arrays

- **numpy.matrix**: a rectangular table of data of the **same** type
  - 2-dimensional
  - \* for matrix multiply
- **numpy.ndarray**:  
 $n$ -dimensional matrix ( $n \geq 1$ )
  - \* is element-wise
  - @ for matrix multiply

|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 1  | 2  | 31 | 2  | 9  | 7  | 34 | 22 | 11 | 5  |
| 11 | 92 | 4  | 3  | 2  | 2  | 3  | 3  | 2  | 1  |
| 3  | 9  | 13 | 8  | 21 | 17 | 4  | 2  | 1  | 4  |
| 8  | 32 | 1  | 2  | 34 | 18 | 7  | 78 | 10 | 7  |
| 9  | 22 | 3  | 9  | 8  | 71 | 12 | 22 | 17 | 3  |
| 13 | 21 | 21 | 9  | 2  | 47 | 1  | 81 | 21 | 9  |
| 21 | 12 | 53 | 12 | 91 | 24 | 81 | 8  | 91 | 2  |
| 61 | 8  | 33 | 82 | 19 | 87 | 16 | 3  | 1  | 55 |
| 54 | 4  | 78 | 24 | 18 | 11 | 4  | 2  | 99 | 5  |
| 13 | 22 | 32 | 42 | 9  | 15 | 9  | 22 | 1  | 21 |

# Data frames (pandas.core.frame.DataFrame)

- Typical data table from a spreadsheet
- Table with rows and columns
- Each column has the same type (e.g. number, text, logical),
- Different columns may have different types

- Example:

|   | localisation | tumorsize | progress |
|---|--------------|-----------|----------|
| 0 | proximal     | 6.3       | False    |
| 1 | distal       | 8.0       | True     |
| 2 | proximal     | 10.0      | False    |

Create the above DataFrame?

```
import pandas as pd
pd.DataFrame({'localisation': ['proximal', 'distal', 'proximal'],
'tumorsize': [6.3, 8.0, 10.0], ...})
```

# Accessing parts of the data

- Rows are also called **observations** or **cases**
- Columns are also called **variables** or **series**
- The data table is an **object**
- Subscripts with `iloc` (“integer location”) in square brackets
  - `y.iloc[0]` identifies first row in matrix/data.frame `y`
  - `y.iloc[0:5]` identifies first 5 rows
  - `y.iloc[:,0]` identifies first column
  - `y.iloc[2,1]` element in row 2 and column 1
  - `y.iloc[1:5,2:4]` elements in rows 1-5 and columns 2-4

Note: row numbers  
(not a column) are not  
included in size

# Subsetting

Note: column/variable  
names are not included  
in size

|    | carat | cut       | color | clarity | depth | table | price | x    | y    | z    |
|----|-------|-----------|-------|---------|-------|-------|-------|------|------|------|
| 1  | 0.52  | Ideal     | D     | VS2     | 61.4  | 56    | 1664  | 5.16 | 5.19 | 3.18 |
| 2  | 0.5   | Very Good | F     | SI1     | 62.3  | 60    | 1250  | 5.07 | 5.11 | 3.17 |
| 3  | 0.61  | Ideal     | G     | VVS2    | 61.6  | 54    | 2242  | 5.45 | 5.49 | 3.37 |
| 4  | 0.36  | Premium   | G     | VS2     | 62.5  | 58    | 756   | 4.55 | 4.51 | 2.83 |
| 5  | 0.7   | Very Good | E     | VS2     | 63.5  | 54    | 2889  | 5.62 | 5.66 | 3.58 |
| 6  | 0.56  | Ideal     | F     | VS1     | 61.7  | 56    | 2016  | 5.32 | 5.28 | 3.27 |
| 7  | 1.19  | Premium   | E     | I1      | 60.2  | 61    | 3572  | 6.91 | 6.87 | 4.15 |
| 8  | 0.52  | Ideal     | F     | IF      | 60.6  | 57    | 2575  | 5.21 | 5.22 | 3.16 |
| 9  | 0.38  | Ideal     | E     | IF      | 62.7  | 55    | 1433  | 4.61 | 4.67 | 2.91 |
| 10 | 0.51  | Ideal     | E     | VS2     | 62.1  | 54    | 1608  | 5.13 | 5.15 | 3.19 |

df.loc[3] row 4

df.iloc[5,6] element in  
row 6 and column 7

df.iloc[:, 3]  
df.clarity  
df["clarity"]

} column 4

df.iloc[7:10, 7:9]  
df.loc[7:10, ("x", "y")]

# Logical Indexing

- `d[d<20]`: extract all elements of `d` that are smaller than 20
- `d<20`
  - A logical vector (True/False)
- Convert from logical to numeric indexes:
  - `np.where()`
  - `x = np.arange(1, 26, step=3)`
  - `x==7: condition`
  - `np.where(x==7)`

# Categoricals

- A **character string** can contain arbitrary text.
- A **categorical** is a variable that can only take a limited number of values, which are called **categories**.
- Limit the **vocabulary**, with a small number of allowed **words**

```
>>> iris.Species.dtype
```

```
CategoricalDtype(categories=['setosa', 'versicolor',  
'virginica'], ordered=False, categories_dtype=object)
```

# The Iris Flower Dataset



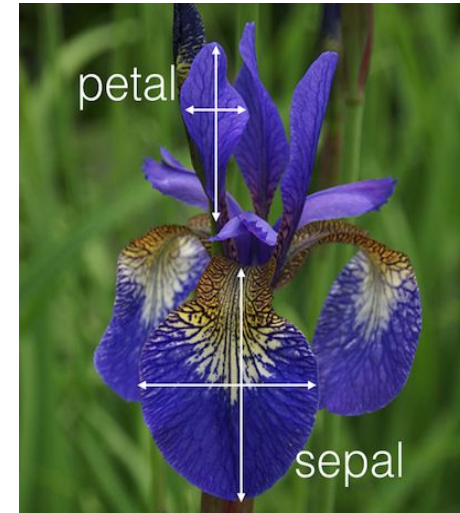
**Iris setosa**



**Iris versicolor**



**Iris virginica**



**Sepal /Petal**

[https://en.wikipedia.org/wiki/Iris\\_flower\\_data\\_set](https://en.wikipedia.org/wiki/Iris_flower_data_set)

[http://sebastianraschka.com/Articles/2014\\_python\\_lda.html](http://sebastianraschka.com/Articles/2014_python_lda.html)



# The iris dataset

- Show first 10 rows
- Show column Sepal.Length
- What is the mean Sepal.Length?
- Show all rows where Sepal.Length > 7.6
  - What are the row indexes where Sepal.Length > 7.6?

# Classwork

- [soodk6/cpsc375-classwork](#)  
–python.ipynb – Part 2

# Coercion

When different objects are mixed in a vector, coercion occurs so that every element in the vector is of the same class:

```
>>> np.array([1,"a"]) ## character  
array(['1', 'a'], dtype='<U11')  
>>> np.array([True,2]) ## numeric  
array([1, 2])  
>>> np.array(["a",True]) ## character  
array(['a', 'True'], dtype='<U5')
```

# Explicit coercion

Objects can be explicitly coerced from one class to another using the **astype** method:

```
>>> x=np.arange(1,6)
```

```
>>> x.dtype  
dtype('int32')
```

```
>>> x.astype(float)  
array([1., 2., 3., 4., 5.])
```

```
>>> x.astype(bool)  
array([ True,  True,  True,  True,  True])
```

```
>>> x.astype(str)  
array(['1', '2', '3', '4', '5'], dtype='<U11')
```

# Some special values

- `np.NaN`: Not a Number (IEEE 754, used by Pandas as placeholder for a missing value).
- `pd.NaT`, `pd.NA` : Not a Time, Not Available (placeholders for missing values).
- `None`: empty value (Python equivalent of C NULL, C++ nullptr)
- `np.Inf`: Infinity (IEEE 754)
- `pd.isna()` and `np.isinf()`
  - to identify missing and infinite values in data
  - Pandas treats `None` as missing
- `df.notna().all(axis='columns')` to identify the rows in a data frame that are complete (no missing values)
- Many functions give NaN if *any* of the inputs is NaN, e.g.: `np.mean(x)`
  - Some functions have `nan*` versions to ignore NaNs, e.g.: `np.nanmean(x)`
  - Pandas methods ignore NA values by default, e.g.: `iris['Sepal.Length'].mean()`
  - Give parameter `skipna=False` to retain the NAs in the input

# Classwork

- [soodk6/cpsc375-classwork](#)  
–python.ipynb – Part 3

# Storing (in memory) variables

- Every Pandas object can be stored into a file with the method “to\_pickle” and restored from a file with the function “read\_pickle”.
- This uses a Python-specific format that is portable between MS-Windows, Linux, Mac.

```
>>> x.to_pickle("x.pkl")
```

```
>>> pd.read_pickle ("x.pkl")
```

# Questions?



menti.com  
9157 9820